

# 操作系统实验文档

---

## Lab 12 - 同步 2 Synchronization 2

---

64d8bdae

Built by github-actions, at 2025-05-06 11:32:50+08:00

# 1. 每周实验

---

## 1.0.1 1.2 Week 12 - 同步 2 Synchronization 2

### 1.2.1 Synchronization 2

#### 1.2.2 实验目的

1. 掌握信号量原理
2. 掌握生产者消费模型

#### DATA RACE

在 Synchronization 1 的课程内容中，我们没有给数据竞争一个精准的定义，我们现在补上它：

Two (or more) memory operations **conflict** if they access the same location and at least one of them is a write operation.

如果多个内存访问在访问同一个地址、并且至少其中一个为写访问，我们称这种情况为数据竞争。

#### 如果我们不让写，那是不是可以并发地读？

这就是 `rwlock` 的设计原理，我们将对共享资源（内存）的访问分为读、写两种情况。`lock` 操作分为 `rdlock` 和 `wrlock`。

在同一时刻，可存在多个读者 (reader) 并发地读（同时从 `rdlock` 返回），或者只能有一个写者 (writer)。单个 writer 和所有 reader 以及其他 writer 互斥。

### **i 如果数据竞争没有副作用 (side-effect)**

假如说，我们允许数据竞争的出现，但是保证它不引起bug，我们就可以避免使用锁了。

RCU (Read-Copy-Update) 是一种同步机制，用于在由指针链接（如链表）的多个对象中避免使用锁，以及降低读者的开销。

指针更新和解引用实际上是无锁的；但是，所有读者(reader)被保证看到的都是旧指针或新指针，不会出现读到一个非法指针的情况；而更新者(updater)则可以在所有能看到旧指针的读者退出后释放 (reclaimer) 旧指针的对象。

In computer science, read-copy-update (RCU) is a synchronization mechanism that avoids the use of lock primitives while multiple threads concurrently read and update elements that are linked through pointers and that belong to shared data structures (e.g., linked lists, trees, hash tables).

Whenever a thread is inserting or deleting elements of data structures in shared memory, all readers are guaranteed to see and traverse either the older or the new structure, therefore avoiding inconsistencies (e.g., dereferencing null pointers).

See also: <https://www.kernel.org/doc/html/next/RCU/whatisRCU.html>

## **SEMAPHORE**

信号量是一种 **带计数器的互斥锁**。

我们可以用“有限个数的共享资源”来理解 Semaphore。例如游泳馆的储物柜、或者停车场的车位。

游泳馆有有限个手环，在进入游泳馆时，给每个人发一个手环（持有 Semaphore）；只有持有手环的人才能进入游泳馆（互斥）；当储物柜被占满时，我们无法给新来的人发手环了，于是他们只能等待（等待）；离开游泳馆的人要还回手环（释放）。

我们可以发现，互斥锁 mutex 是 Semaphore 中  $N = 1$  的特殊情况。

信号量有两种主要行为：请求和释放。当信号量有剩余时，则请求的结果是持有信号量直到释放；当信号量没有剩余时，则请求的结果是等待，直到有多余的信号量释放出来则变为持有信号量。

我们定义一套原语：`sem_acquire()` / `sem_release()`（也可以写作 `P()` / `V()`、`wait()` / `post()`、`decrease()` / `increase()`）：

1. Semaphore 拥有一个初始值。
2. 在 `acquire` 时 Semaphore 的值减 1，在 `release` 时 Semaphore 的值加 1。
3. Semaphore 的值永远不会低至 0 以下。如果值为 0，`acquire` 将会阻塞(blocking)并等待；另一个 `release` 会唤醒它并使它尝试 `acquire`。

`sem_wait()` decrements (locks) the semaphore pointed to by `sem`. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until it becomes possible to perform the decrement (i.e., the semaphore value rises above zero).

`sem_post()` increments (unlocks) the semaphore pointed to by `sem`. If the semaphore's value consequently becomes greater than zero, then another process or thread blocked in a `sem_wait(3)` call will be woken up and proceed to lock the semaphore.

另一个额外的性质：`acquire` (`wait`) 可能会导致等待（阻塞），而 `release` (`post`) 永远不会导致等待（阻塞）。

我们将在最后一次作业中，在 xv6 上实现信号量，并实现哲学家吃饭问题。

#### 生产者-消费者模型

Producer-Consumer 模型是同步问题和并发编程中非常常见的模型。

生产者 (Producer) 负责产生数据或任务；消费者 (Consumer) 负责处理。二者通过缓冲区 (Buffer) 或队列 (Queue) 隔离，互不阻塞。生产者和消费者只需要在“访问缓冲区”时进行互斥，保护缓冲区的合法性即可。

我们可以写出来 Producer 和 Consumer 的同步条件：

- Producer (生产数据)：如果缓冲区有空位，放入；否则等待（阻塞）
- Consumer (消费数据)：如果缓冲区有数据，取走；否则等待（阻塞）

在 Producer 和 Consumer 之间，我们完成了一种同步：同一个 object 的生产必须 happens-before 消费。

通常来说，缓冲区一般是固定大小的数组（一旦创建后不再扩容），例如 Ring Buffer。

## 条件变量

有了同步条件后，我们可以使用上次课程中学习的条件变量来完成同步：同步的对象 (chan) 为 `&buf`，保护它的 spinlock 为 `&buf.lock`；每当往 buffer 中放置/取走数据后，我们使用 `wakeup` 通知所有在该对象上 `sleep` 的线程。

```
struct buffer {
    spinlock_t lock;
    // ...
} buf;

bool is_full(struct buffer* b);
bool is_empty(struct buffer* b);
void put_data(struct buffer* b, void* data);
void* get_data(struct buffer* b);

void produce(void* data) {
    acquire(&buf.lock);
    while(is_full(&buf))
        sleep(&buf, &buf.lock);

    assert(!is_full(&buf));
    put_data(&buf, data);    // modify the buffer, guarded by buf.lock and cond: !full

    wakeup(&buf);
    release(&buf.lock);
}

void* consume(void* data) {
    acquire(&buf.lock);
    while(is_empty(&buf))
        sleep(&buf, &buf.lock);

    assert(!is_empty(&buf));
    void* data = get_data(&buf);    // modify the buffer, guarded by buf.lock and cond: !empty

    wakeup(&buf);
    release(&buf.lock);

    return data;
}
```

具体实现可以参照 xv6lab10 中的 `sync_main.c` 文件。你可以尝试注释掉 `producer` 和 `consumer` 中的条件检查，观察是否能够通过检查。

```
while (is_empty(&buf)) sleep(&buf, &buf.lock);
assert(!is_empty(&buf));
```

## Semaphore

我们也可以使用信号量来实现生产者-消费者模型：

1. Producer/Consumer 分别需要在两个事件上等待：Buffer 为空或者为满。所以，我们创建两个信号量 `avail` 和 `empty`，表示当前 Buffer 有多少个元素 以及 有多少个空位，它们的初始值为 0 和 N。
2. 在 `produce` 中 `wait(empty)`，这表示 有多少空位 --，即我们等待一个空位。拿到空位后，`produce` 可以向 Buffer 中添加一个对象；然后 `post(avail)`，这表示 有多少元素 ++。
3. 相反的，在 `consume` 中 `wait(avail)`，这表示 有多少个元素 --，即我们等待一个元素；以及 `post(empty)` 这表示 有多少空位 ++。
4. 最后，我们在访问 Buffer 时需要互斥，所以我们需要一个 `mutex` 信号量，其初始值为 1。

```
struct buffer {
    sem_t mutex;
    sem_t avail;
    sem_t empty;
    // ...
} buf;

void put_data(struct buffer* b, void* data);
void* get_data(struct buffer* b);

void produce(void* data) {
    wait(&buf.empty);    // empty space --

    wait(&buf.mutex);
    put_data(&buf, data);    // critical section for buf.
    post(&buf.mutex);

    post(&buf.avail);    // avail ++
}

void* consume(void* data) {
    wait(&buf.avail)    // avail --

    wait(&buf.mutex);
    void* data = get_data(&buf);    // critical section for buf.
    post(&buf.mutex);

    post(&buf.empty);    // empty space ++

    return data;
}
```

## 不变量 INVARIANT

什么是不变量 Invariant：不变量（invariant）是在程序执行的 **任何时刻都必须成立的断言(assert)**。

我们可能在数理逻辑课程上学习过 循环不变量 (loop invariant) 的概念，它表示在一个循环中永远成立的一个条件，通常可以用于证明一个循环的性质。

在并发编程中，某些复杂的同步问题中会涉及到多个共享变量之间的关系。不变量就是几个变量之间的逻辑关系。假如我们使用两个 `int` 类型的变量 `avail` 和 `empty` 表示一个定长的 Ring Buffer 有多少个对象、多少个空余对象。那么它的不变量就是 `avail + empty == N`。

那么，如果我们有时候需要打破不变式，那么我们就使用 Critical Section 将其保护起来，使得其他 CPU 永远不能观测到该不变式被打破了，这也即是我们在之前的课程中提到的：

我们可以将 (A, B) 打包 **一个不可中断的整体**。即，在其他CPU的视角下，这两个事件是在一瞬间就发生完了的（即原子的 (Atomic)）。也就是说，其他CPU不可能看到这个整体的中间状态。

通常来说，我们在 Critical Section 中会访问、修改多个共享变量，但是他们在 Critical Section 之外恒定地满足某种关系。

### 1.2.3 Linux pthread API

pthread (posix thread) 库是 Linux 下的多线程库。它有以下几组 API：

1. 创建线程： `pthread_create`，等待线程退出： `pthread_join`。
2. 互斥锁 mutex（一种 sleeplock）： `pthread_mutex_t`

```
// man pthread_mutex_init

#include <pthread.h>

pthread_mutex_t fastmutex = PTHREAD_MUTEX_INITIALIZER;

int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);

int pthread_mutex_lock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

DESCRIPTION A mutex is a MUTual EXclusion device, and is useful for protecting shared data structures from concurrent modifications, and implementing critical sections and monitors.

A mutex has two possible states: unlocked (not owned by any thread), and locked (owned by one thread). A mutex can never be owned by two different threads simultaneously. A thread attempting to lock a mutex that is already locked by another thread is suspended until the owning thread unlocks the mutex first.

### 1. 条件变量: `pthread_cond_t`

```
// man pthread_cond_init

#include <pthread.h>

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);
int pthread_cond_destroy(pthread_cond_t *cond);

int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```



```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);
```

A condition (short for "condition variable") is a synchronization device that allows threads to suspend execution and relinquish the processors until some predicate on shared data is satisfied. The basic operations on conditions are: signal the condition (when the predicate becomes true), and wait for the condition, suspending the thread execution until another thread signals the condition.

A condition variable must always be associated with a mutex, to avoid the race condition where a thread prepares to wait on a condition variable and another thread signals the condition just before the first thread actually waits on it.

`pthread_cond_init` initializes the condition variable `cond`, using the condition attributes specified in `cond_attr`, or default attributes if `cond_attr` is `NULL`. Variables of type `pthread_cond_t` can also be initialized statically, using the constant `PTHREAD_COND_INITIALIZER`.

`pthread_cond_signal` restarts one of the threads that are waiting on the condition variable `cond`. If no threads are waiting on `cond`, nothing happens. If several threads are waiting on `cond`, exactly one is restarted, but it is not specified which.

`pthread_cond_broadcast` restarts all the threads that are waiting on the condition variable `cond`. Nothing happens if no threads are waiting on `cond`.

`pthread_cond_wait` atomically unlocks the mutex (as per `pthread_unlock_mutex`) and waits for the condition variable `cond` to be signaled. The thread execution is suspended and does not consume any CPU time until the condition variable is signaled. The mutex must be locked by the calling thread on entrance to `pthread_cond_wait`. Before returning to the calling thread, `pthread_cond_wait` re-acquires mutex (as per `pthread_lock_mutex`).

Unlocking the mutex and suspending on the condition variable is done atomically. Thus, if all threads always acquire the mutex before signaling the condition, this guarantees that the condition cannot be signaled (and thus ignored) between the time a thread locks the mutex and the time it waits on the condition variable.

## 1. Semaphore

```
#include <semaphore.h>

// man sem_init
int sem_init(sem_t *sem, int pshared, unsigned int value);

// man sem_post
```

```
int sem_post(sem_t *sem);

// man sem_wait
int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_timedwait(sem_t *restrict sem, const struct timespec *restrict abs_timeout);
```

### 1.2.4 Lab 练习

1. 假设有两种线程，每种线程若干个：第一种线程死循环地打印左括号 (，第二种线程死循环地打印右括号 )。现在要求：在所有时刻，打印出来的字符串是平衡的括号字符串或其前缀。如 ()(()) 和 ((())。最多允许 N 层嵌套括号。

例如：

- ()(()) 是一个合法的平衡括号，最大嵌套深度为 3。
- () 不是一个合法的平衡括号。
- ((( 是 (((()) 的前缀，嵌套深度为 2。

已知有一个变量 `count` 表示：目前左括号比右括号多了几个。请你写出这两种线程的同步条件。即，每种线程在什么时候可以打印 ( 或 )。

2. 在使用条件变量实现生产者-消费者模型时，`produce` 方法中的 `while()` `sleep` 循环可以换成 `if()` `sleep` 吗？

```
void produce(void* data) {
    acquire(&buf.lock);
    if (is_full(&buf))          // change while to if
        sleep(&buf, &buf.lock);

    assert(!is_full(&buf));
    put_data(&buf, data);      // modify the buffer, guarded by buf.lock and cond: !full

    wakeup(&buf);
    release(&buf.lock);
}
```

3. 使用 Linux 命令 `man xxx` 查阅 `pthread_cond_signal` 和 `pthread_cond_broadcast` 的 manpage，简述它们之间的区别。

并回答：在生产者-消费者模型的实现中，我们应该用几个条件变量 (`pthread_cond_t`)？以及我们在通知条件变更 (xv6中的 `wakeup`) 时应该使用 `pthread_cond_signal` 还是 `pthread_cond_broadcast`？